Chapter 7 — The Box Model and Layout Basics

Chapter 6 taught you how the browser decides *which* declarations apply to an element. This chapter is about *where the element ends up*. Everything on a web page is a rectangle — a **box** — and layout is the job of sizing those boxes and placing them. Before Flexbox (Chapter 8, for laying items out in a row or column) and Grid (Chapter 9, for rows *and* columns at once) make sense, you need the rules they build on: how a box's size is calculated, how boxes flow down the page by default, how you pull a box out of that flow with `position`, what happens when content does not fit, and who is painted on top of whom.

What you'll learn

#	Topic	You will be able to…
7.1	The box model	name the four areas of a box and predict its rendered size
7.2	`box-sizing`	explain why almost every project starts with `border-box`
7.3	Margin collapsing	predict the gap between two blocks and stop margins leaking out of a parent
7.4	`display`	choose between block, inline, inline-block, none, flow-root and contents
7.5	Normal flow and formatting contexts	describe how boxes are laid out when you do nothing, and what a BFC (block formatting context) is
7.6	Positioning	use relative, absolute, fixed and sticky and know each one's reference box
7.7	Overflow	pick between visible, hidden, clip, scroll and auto; truncate text with an ellipsis (the "…" that shows text was cut off)
7.8	`z-index` and stacking contexts	explain why `z-index: 9999` (the property that decides which overlapping box is in front) sometimes "doesn't work"
7.9	Floats (legacy — old, kept for compatibility)	wrap text around an image, contain floats, and read old float layouts

Every example is a small page in `code/` with a screenshot in `img/`. The outlines and labels you see are drawn by the example CSS so the boxes are visible. The `.mjs` scripts open the same pages in real headless Chrome (Chrome without a window, driven by a script via `puppeteer-core`, a Node.js library for controlling Chrome; the same pattern as Chapter 1's network script) and print the sizes the browser actually computed with `getBoundingClientRect()` (a DOM method that returns an element's on-screen position and size) — so every number in this chapter is measured, not guessed. Run one with `node code/02_margin_collapse.mjs`.

> **Tip — outlines are your layout debugger.** An `outline` is drawn *around* a box but takes up no space, so adding `outline: 1px solid red` never changes the layout you are inspecting (a `border` would). Many developers keep a snippet (a short reusable piece of code) like `* { outline: 1px solid rgb(255 0 0 / .3) }` handy for exactly this.

7.1 The box model

What it is

The **CSS box model** says every element generates a rectangular box made of four nested areas, from the inside out:

Area	What it is	Set with	Takes the element's background?
Content	Where text, images and child boxes go	`width`, `height` (and `min-`/`max-` versions)	Yes
Padding	Breathing room between the content and the border	`padding`, `padding-top` …	Yes
Border	A line (or image) around the padding	`border`, `border-width`, `border-style`, `border-color`	Drawn on top of the background
Margin	Empty space *outside* the border that pushes other boxes away	`margin`, `margin-left` …	No — always transparent

The outer edge of each area has a name used all over the specs: **content box**, **padding box**, **border box**, **margin box**.

Analogy: a framed photo on a wall. The photo is the content, the white mat around it is padding, the frame is the border, and the gap you leave to the next frame is the margin.

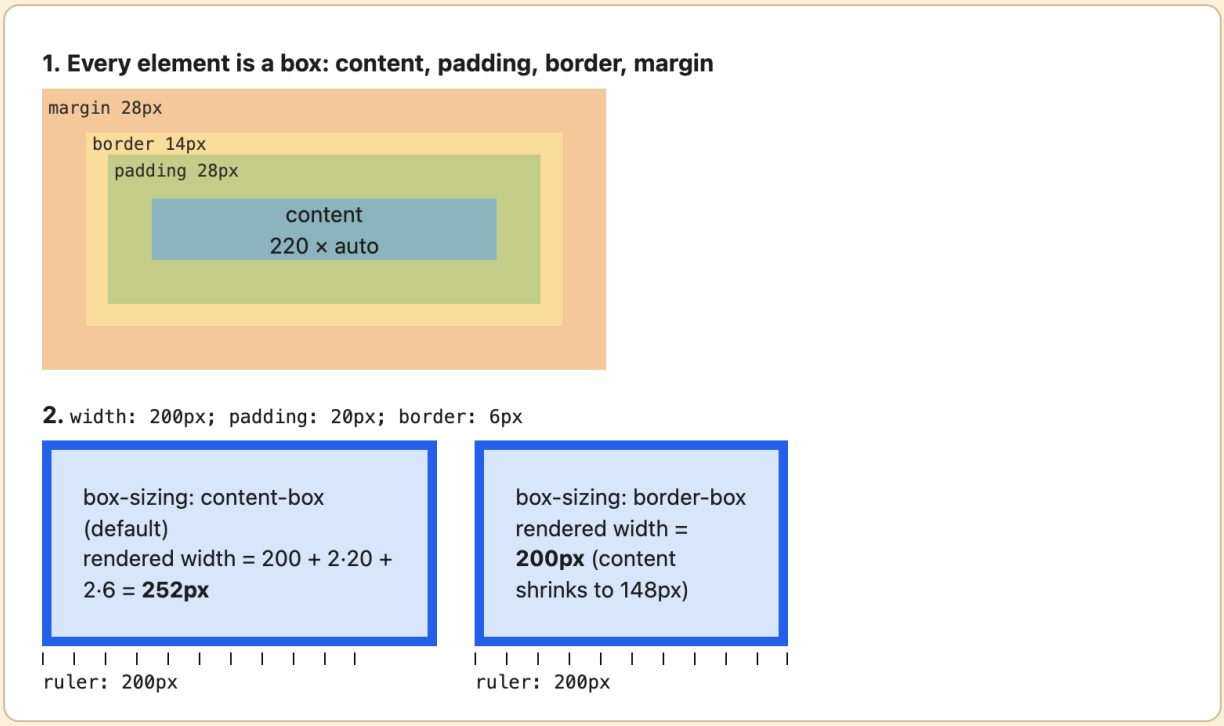


Figure 7.1 — `code/01_box_model.html`. The top diagram uses the same colours as the Chrome/Edge DevTools (the browser's built-in developer tools, F12) box-model panel (Elements → Computed). The blue content and green padding come from two background layers clipped to `content-box` and `padding-box`.

How the size is calculated

With the default `box-sizing: content-box`, `width` and `height` set the **content box only**. The space the element occupies is:

```
rendered width  = width  + padding-left + padding-right + border-left + border-right
space taken     = rendered width + margin-left + margin-right
```

So `width: 200px; padding: 20px; border: 6px solid` gives 200 + 2×20 + 2×6 = **252px** on screen. That surprise is exactly what `box-sizing` (7.2) fixes.

Shorthands — the clock rule

`margin`, `padding`, `border-width` and friends take 1–4 values that go **clockwise from the top**:

Written	Top	Right	Bottom	Left
`margin: 10px`	10	10	10	10
`margin: 10px 20px`	10	20	10	20
`margin: 10px 20px 30px`	10	20	30	20
`margin: 10px 20px 30px 40px`	10	20	30	40

`border: 2px solid #333` sets width, style and colour of all four sides; the style is required — `border: 2px #333` draws nothing because the default style is `none`. Modern code often uses **logical properties** such as `margin-block` (top + bottom in horizontal text) and `padding-inline` (left + right), which follow the writing direction (left-to-right, right-to-left or vertical, depending on the language); they are covered in Chapter 13.

Sizing rules worth knowing

- **`width: auto`** (the default) on a block box means "fill the containing block's width, minus my margins, borders and padding". The **containing block** is the rectangle a box is sized and positioned against — for ordinary boxes, the parent's content box (§7.6 gives the full rules). It is *not* the same as `width: 100%`, which ignores margins and can overflow.
- **`height: auto`** means "as tall as my content". A percentage height (`height: 50%`) only works if the parent has a definite height; otherwise it behaves like `auto`.
- **Percentages for padding and margin — even top and bottom — refer to the containing block's *width*.** That is why the old `padding-top: 56.25%` trick made 16:9 boxes (today, use `aspect-ratio: 16 / 9`).
- **`min-width` / `max-width`** clamp the box: `max-width: 65ch` keeps text readable; `min-height: 100vh` makes a full-screen section that can still grow. If they conflict, `min-*` wins.
- **Intrinsic sizes**: `width: min-content` (as narrow as the longest word allows), `max-content` (as wide as the content wants with no wrapping), and `fit-content` (max-content, but never wider than the available space). You will meet these again as flex and grid track sizes (a *track* is one row or column of a grid).
- **Margins can be negative** (pull a box or its neighbours closer); **padding cannot**.
- **`margin: auto`** on a block with a set width splits the leftover horizontal space equally — the classic `margin: 0 auto` horizontal centring.

Border vs outline vs box-shadow

	Takes space?	Per side?	Follows `border-radius`?	Typical use
`border`	Yes	Yes	Yes	Visible frames, dividers
`outline`	No (drawn outside the border box)	No	Yes in current browsers	Focus rings (the outline showing which element has keyboard focus, Chapter 3), debugging
`box-shadow`	No	No (but can be offset)	Yes	Elevation (making a card look raised off the page), fake borders that don't shift layout

Common mistakes

- Setting `width: 100%` *and* padding on a content-box element → it overflows its parent by the padding + border.
- Expecting `height: 100%` to fill the screen when the parent's height is `auto`.
- Writing `border: 1px red` and wondering why nothing shows (no style).

Self-check

1. A content-box element has `width: 300px; padding: 10px 20px; border: 2px solid`. How wide is it on screen? → 300 + 40 + 4 = **344px**.

2. `padding-top: 10%` inside a 600px-wide, 200px-tall parent — how tall is that padding? → **60px** (percentages use the width).
3. Which of border/outline changes the layout? → Only `border`.

7.2 `box-sizing`

What it is

`box-sizing` decides *which box* `width` and `height` describe:

Value	`width` means…	Padding and border…
`content-box` (default)	the content box	are **added** on top
`border-box`	the border box	are **taken out of** the width; the content shrinks

The problem it solves

Designers think in outer sizes: "the sidebar is 300px wide". With content-box, adding 20px padding secretly makes it 340px and a carefully measured two-column layout breaks. With border-box, 300px stays 300px whatever padding you add later. The measured result for the two boxes in Figure 7.1:

```
// code/01_box_sizing.mjs (key part) — both boxes: width: 200px; padding: 20px; border: 6px
const rows = await withPage('01_box_model.html', page => page.$$eval('.demo', els => els.map(el => {
  const cs = getComputedStyle(el);
  const content = el.clientWidth - 2 * parseFloat(cs.paddingLeft);   // clientWidth = content + padding
  return `${cs.boxSizing}  width property ${cs.width}  content ${content}px  ` +
         `on screen (border box) ${el.getBoundingClientRect().width}px`;
})));
```

Output:

```
content-box  width property 200px  content 200px  on screen (border box) 252px
border-box   width property 200px  content 148px  on screen (border box) 200px
```

`withPage` is a ten-line helper in `code/measure.mjs`: launch headless Chrome, open the file, run a function in the page, close. The three DOM measurements you will use most:

API	Returns
`el.getBoundingClientRect()`	The border box's position and size on screen (includes transforms — CSS moves, rotations and scaling — and fractional pixels such as 12.5)
`el.offsetWidth` / `offsetHeight`	Border box size, rounded to an integer, ignores transforms
`el.clientWidth` / `clientHeight`	Padding box size minus any scrollbar (the bar you drag to scroll)

The universal reset

Almost every modern stylesheet (and every CSS reset — a starter stylesheet that removes or evens out browser defaults, Chapter 13) starts with:

```
*, *::before, *::after { box-sizing: border-box; }
```

`box-sizing` is not inherited, so the rule targets every element and pseudo-element directly. A variant sets `html { box-sizing: border-box }` and `*, *::before, *::after { box-sizing: inherit }` so a third-party widget (a ready-made component from another vendor) can opt back into content-box for its whole subtree (the element and everything inside it).

Browsers' default styles already give many form controls border-box sizing (in Chrome, `<input>`, `<select>`, `<button>` and `<textarea>` all compute to `border-box`), and browsers differ on some of them — one more reason to set it for everything yourself, so a `div` and an `input` with the same `width` line up.

When not to use it

There is essentially no downside to border-box. The one thing to remember: if padding + border are larger than the width (say `width: 20px; padding: 20px`), the content box cannot go below zero, so the box grows beyond `width` anyway.

In the interview

> **30-second answer:** "Every element is a box of content, padding, border and margin. By default `width` sets only the content, so padding and border are added on top — 200 wide plus 20 padding plus 6 border is 252. With `box-sizing: border-box`, `width` includes padding and border, so the box stays 200 and the content shrinks. Most projects apply border-box to everything in the reset because it matches how designs are measured. Margin is never included in either model."

- *Is margin part of the element's width under border-box?* → No. Margin is always outside.
- *Is `box-sizing` inherited?* → No, hence the `*` selector (or the `inherit` pattern).

Self-check

1. Border-box, `width: 250px; padding: 25px; border: 5px`. Content width? → 250 − 50 − 10 = **190px**.
2. Why include `*::before, *::after` in the reset? → Pseudo-elements (CSS-generated boxes, Chapter 6) are boxes too and `*` alone does not match them.

7.3 Margin collapsing

What it is

In normal block layout, when two **vertical** margins touch with nothing between them, they **collapse** into a single margin whose size is the *larger* of the two instead of their sum. Horizontal margins never collapse.

Analogy: two people who each want "at least 30 cm of personal space" stand 30 cm apart, not 60.

Why it matters

It is the rule behind the most common "why is there a gap there?" and "why did my margin disappear?" questions. It was designed for text: a paragraph with `margin: 1em 0` followed by another should be 1em apart, not 2em.

How it works — the three situations

1. **Adjacent siblings** (elements with the same parent, one right after the other): the bottom margin of one block and the top margin of the next collapse.
2. **Parent and first/last child**: if a parent has no border, padding, inline content or clearance (extra space added by `clear` below a float, §7.9) separating it from its first child, the child's top margin collapses *through* the parent — it appears **outside** the parent. Same for the last child's bottom margin (when the parent's height is `auto`).
3. **Empty blocks**: a block with no content, padding, border or height has its own top and bottom margins collapse together.

The arithmetic

Margins meeting	Result
All positive (30, 20)	The largest: **30**
Positive and negative (30, −10)	Largest positive + most negative: 30 + (−10) = **20**
All negative (−10, −25)	The most negative: **−25**

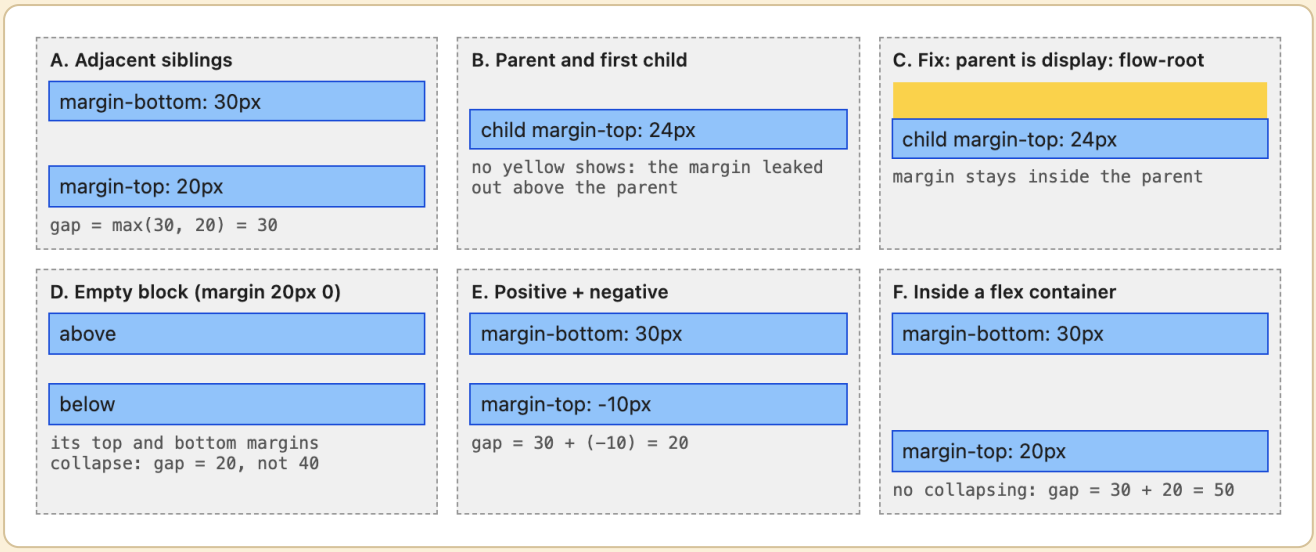


Figure 7.2 — `code/02_margin_collapse.html`. In case B no yellow is visible at all: the 24px margin sits above the parent, and the child fills the parent exactly.

The measured gaps:

```
// code/02_margin_collapse.mjs (key part)
const gap   = (a, b) => r(b).top - r(a).bottom;                // space between two boxes
const inset = id => { const p = document.getElementById(id);    // child top minus parent top
  return p.firstElementChild.getBoundingClientRect().top - p.getBoundingClientRect().top; };
```

Output:

```
A siblings 30 / 20           -> 30px
B child top inside parent    -> 0px
C same with flow-root parent -> 24px
D across empty block 20/20   -> 20px
E 30 / -10                   -> 20px
F flex column 30 / 20        -> 50px
```

What stops margins collapsing

- Anything **between** them: padding or border on the parent (even `padding-top: 1px`), inline content, or a `clear`ed float.
- The parent starting a new **block formatting context** (7.5): `display: flow-root`, `overflow` other than `visible`/`clip`, `display: inline-block`, floats, absolute positioning.
- **Flex and grid layouts**: margins of flex and grid items (the direct children of a flex or grid container) never collapse (case F). This is one reason many teams lay out vertical stacks with `display: flex; flex-direction: column; gap: …` or `display: grid; gap: …` — `gap` is predictable and never collapses.
- Floated and absolutely positioned boxes never collapse with anything.

Common mistakes

- Giving a card's first heading `margin-top` and seeing the *whole card* move down (parent-child collapse). Fix: `display: flow-root` or padding on the card, or remove the child's top margin.
- Setting `margin-bottom: 20px` on every list item *and* `margin-top: 20px` on the next and expecting 40px.
- Trying to "fix" collapsing with `overflow: hidden` — it works (new BFC) but also clips shadows and focus rings. Prefer `flow-root`.

Real-world examples

Many design systems (a team's shared library of components and style rules — and the "Every Layout" book's *Stack* idea) avoid the whole topic by setting margins in **one direction only** — for example `.stack > * + * { margin-block-start: 1rem }` — or by using `gap`.

In the interview

> **30-second answer:** "Vertical margins between blocks in normal flow collapse to the larger one — 30 and 20 give 30, not 50. It happens between siblings, between a parent and its first or last child when nothing separates them, and on empty blocks. Negative margins are added to the largest positive one. It doesn't happen in flex or grid, for floats or absolutely positioned boxes, or when padding, a border or a new block formatting context like `display: flow-root` separates the margins."

- *Why did my `h2`'s margin push the whole section down?* → Parent-child collapse; add padding or `flow-root` to the section.
- *Do horizontal margins collapse?* → Never.

Self-check

1. Margins 40px and 25px between siblings → gap? → **40px**.
2. −20px and −5px? → **−20px**.
3. The same two items inside a `display: grid` container with no gap, margins 40 and 25? → **65px**, no collapsing in grid.
4. One-line fix to keep a child's margin inside its parent? → `display: flow-root` on the parent.

7.4 `display`

What it is

`display` answers two questions about a box:

- **Outer display type** — how the box itself takes part in its parent's layout: as a **block** (own line) or **inline** (flows within a line of text).
- **Inner display type** — how it lays out its *children*: normal **flow**, **flex**, **grid**, **table**…

Every familiar keyword is a combination. Since 2023 all major browsers (Baseline) also accept the explicit **two-value syntax**:

Keyword	Two-value form	Outer	Children laid out as
`block`	`block flow`	block	normal flow
`inline`	`inline flow`	inline	normal flow (inline)
`inline-block`	`inline flow-root`	inline	flow, in its own formatting context
`flow-root`	`block flow-root`	block	flow, in its own formatting context
`flex`	`block flex`	block	flex (Ch. 8)
`inline-flex`	`inline flex`	inline	flex
`grid` / `inline-grid`	`block grid` / `inline grid`	block / inline	grid (Ch. 9)
`list-item`	`block flow list-item`	block	flow, plus a `::marker`
`table`, `table-row`, `table-cell`	—	—	table layout (what `<table>` uses)

Two keywords generate **no box of their own**:

- `display: none` — the element and its whole subtree produce no boxes: no space, not painted, not in the accessibility tree (the version of the page screen readers read), not focusable. (Compare `visibility: hidden` and `opacity: 0`, which keep the space — measured in Chapter 1, §1.7.)
- `display: contents` — the element's own box disappears but its **children** are laid out as if they were children of the grandparent. Handy for making a wrapper `<div>` or `<ul>`'s items take part in a parent's flex or grid layout. Caution: browsers historically dropped the element's semantics (its meaning, such as "this is a list") from the accessibility tree (a `<ul>` with `display: contents` stopped being announced as a list); most engines have fixed the common cases, but test with a screen reader and avoid it on buttons and other interactive elements.

Block vs inline vs inline-block

Behaviour	`block`	`inline`	`inline-block`
Starts on a new line	Yes	No	No
Default width	Fills the container	Its content	Its content (shrink-to-fit: as narrow as its content allows)
`width` / `height`	Respected	**Ignored**	Respected
Horizontal padding/border/margin	Push neighbours	Push neighbours	Push neighbours
Vertical padding/border	Push neighbours	**Painted but don't move lines** (overlap)	Push neighbours
Vertical margin	Applies (may collapse)	**Ignored**	Applies (never collapses)
Default for	`div`, `p`, `h1`–`h6`, `ul`, `section`, `form`	`span`, `a`, `strong`, `em`, `code`	nothing by default; `button`, `input`, `img` behave similarly

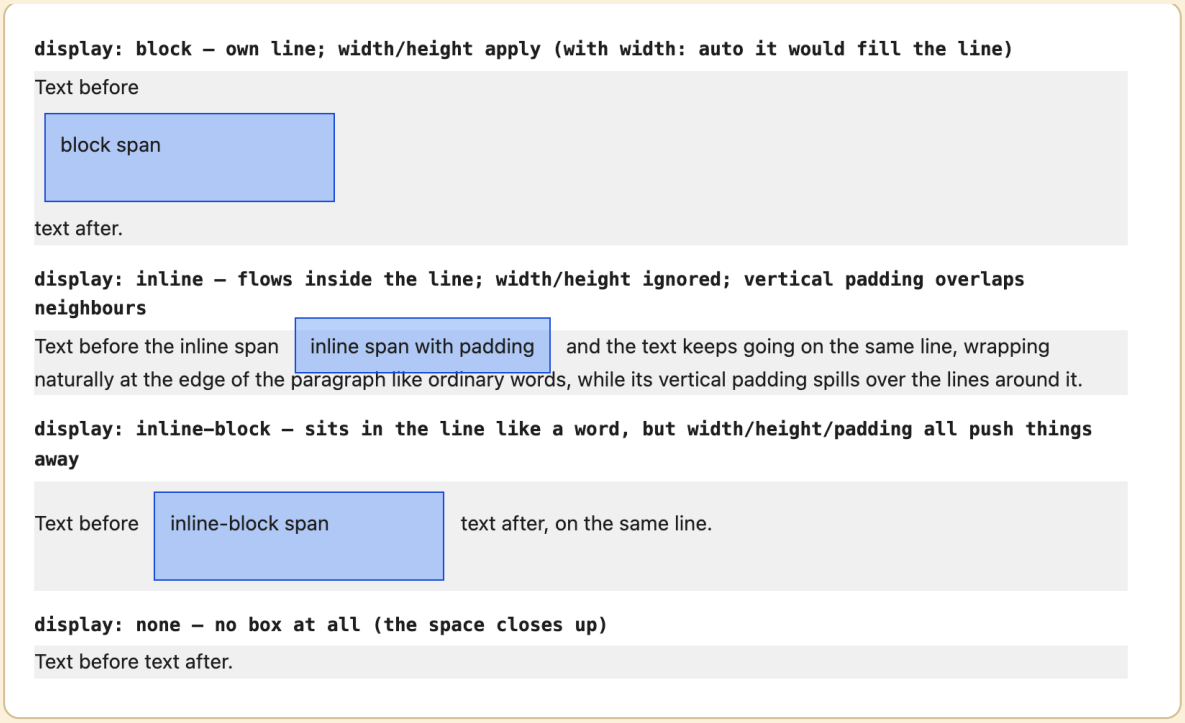


Figure 7.3 — `code/03_display.html`. Every blue span has exactly the same `width: 180px; height: 40px; padding: 10px; margin: 8px`; only `display` differs.

```
block         rendered 200 x 60  height of its <p>: 121
inline        rendered 176 x 37  height of its <p>: 45
inline-block  rendered 200 x 60  height of its <p>: 76
none          rendered   0 x  0  height of its <p>: 22
```

Output of `code/03_display.mjs`. The inline span ignores `width: 180px` (its 176px is just text + padding) and its paragraph is only two ordinary lines tall (45px) despite the 10px vertical padding. The inline-block keeps its 200×60 border box and the line grows to fit it.

Which elements are block or inline by default is only a **user-agent stylesheet default** (the browser's own built-in CSS) — HTML itself defines content models (rules for what may go inside what), not layout (see Chapter 2). A `<span>` with `display: block` is still phrasing content (text-level content) for validity purposes.

Replaced elements

`<img>`, `<video>`, `<iframe>`, `<input>` are **replaced elements**: their content comes from outside CSS, they have an **intrinsic size**, and they are inline-level by default yet accept `width` and `height`. The classic surprise: an `<img>` inside a `div` leaves a few pixels of gap underneath, because as an inline box it sits on the text **baseline** (the invisible line letters sit on) and leaves room for letter descenders (the tails of g, p, y that hang below it). Fix with `img { display: block }` (common in resets) or `vertical-align: middle`.

Common mistakes

- Setting `width` on an `<a>` or `<span>` and seeing no change → make it `inline-block` or `block`.
- Adding `margin-top` to an inline link to push it down → vertical margins on inline boxes are ignored.
- Using `inline-block` for rows of cards and fighting the whitespace gap between them (the HTML spaces between tags render as a space character). Use flex or grid with `gap`.
- Hiding content that should stay available to screen readers with `display: none`; use a *visually-hidden* class instead (Chapter 3).

Self-check

1. Which `display` value would you give an `<a>` that must be 44px tall but stay in a line of text? → `inline-block` (or `inline-flex`).

2. What does `display: inline flex` mean? → The box is inline on the outside, lays its children out with flexbox inside — same as `inline-flex`.
3. Why does an image in a div have a small gap below it? → It is inline and aligned to the text baseline; make it `display: block`.

7.5 Normal flow and formatting contexts

What it is

Normal flow is the layout you get when you write no layout CSS at all. Its rules are few:

1. **Block-level boxes** stack one after another in the **block direction** (top to bottom in English), each starting on a new line and, if `width: auto`, filling the container's width.
2. **Inline-level boxes** (text, `span`, `a`, images) sit side by side in the **inline direction** (left to right in English) and wrap into **line boxes** (one invisible box per line of text) when a line is full.
3. Vertical margins between blocks collapse (7.3).
4. A block's height grows to fit its content.

"Block direction" and "inline direction" are used instead of "vertical" and "horizontal" because they rotate with the writing mode (the CSS setting for text direction, horizontal or vertical): in vertical Japanese text, blocks stack right-to-left and lines run top to bottom. This is why modern properties are named `margin-block`, `inline-size` and so on.

Formatting contexts

A **formatting context** is a region in which one set of layout rules applies to the children. The ones you will meet:

Context	Created by	Children laid out by
Block formatting context (BFC)	root `<html>`, `display: flow-root`, `inline-block`, floats, `position: absolute/fixed`, `overflow` other than `visible`/`clip`, table cells, flex/grid items, `contain: layout/paint` (a performance hint isolating an element's layout/painting), multi-column containers (text flowing into newspaper-style columns)	block and inline flow
Inline formatting context (IFC)	a block that contains only inline-level content	line boxes
Flex formatting context	`display: flex / inline-flex`	flexbox (Ch. 8)
Grid formatting context	`display: grid / inline-grid`	grid (Ch. 9)

A new **BFC** is a sealed room. Inside it:

- child margins do not collapse through its edges (7.3),
- it **contains its floats** — its height includes them (7.9),
- it does not overlap floats outside it; it sits beside them instead.

`display: flow-root` exists *only* to create a BFC with no side effects, which is why it replaced the old `overflow: hidden` and "clearfix" hacks (workarounds, shown in §7.9). It is supported in every current browser.

Centring in normal flow

Goal	Normal-flow technique
Centre inline content (text, inline images) horizontally	`text-align: center` on the parent
Centre a block with a known or max width	`max-width: 40rem; margin-inline: auto`
Centre a block's children vertically	`align-content: center` on the block itself (works on plain blocks since 2024 — Baseline newly available)
Centre anything both ways	Flexbox or grid: `display: grid; place-items: center` (Chapter 9)

Self-check

1. In a vertical writing mode, which direction do block boxes stack? → Horizontally (the block axis is now horizontal).
2. Name two things a new BFC does. → Stops margin collapse through its edges; contains floats; doesn't overlap outside floats.
3. What is the side-effect-free way to create a BFC? → `display: flow-root`.

7.6 Positioning

What it is

The `position` property lets you move a box away from where normal flow would put it, using the **inset properties** `top`, `right`, `bottom`, `left` (or the shorthand `inset: top right bottom left`, same clock rule as `margin`).

Value	Stays in flow (keeps its space)?	Offsets measured from	Scrolls with the page?	Typical use
`static` (default)	Yes	— insets ignored	Yes	Everything
`relative`	**Yes** — the original space is kept	Its own normal position	Yes	Small nudges; anchor for absolute children
`absolute`	**No**	Nearest **positioned** ancestor's (an element above it in the tree whose `position` is not `static`) padding box (else the initial containing block)	Yes, with that ancestor	Badges, dropdowns, overlays (boxes laid on top of other content) inside a card
`fixed`	No	The **viewport** (the visible browser area)	**No** — stays put	Sticky headers of old, cookie bars, chat buttons
`sticky`	Yes	Its nearest **scrolling ancestor** (scrollport — the visible window of a scrollable box)	Scrolls until the threshold, then sticks	Table headers, section headings, sidebars

A **positioned element** is any element whose `position` is not `static`. The rectangle an element's offsets and percentages refer to is its **containing block**.

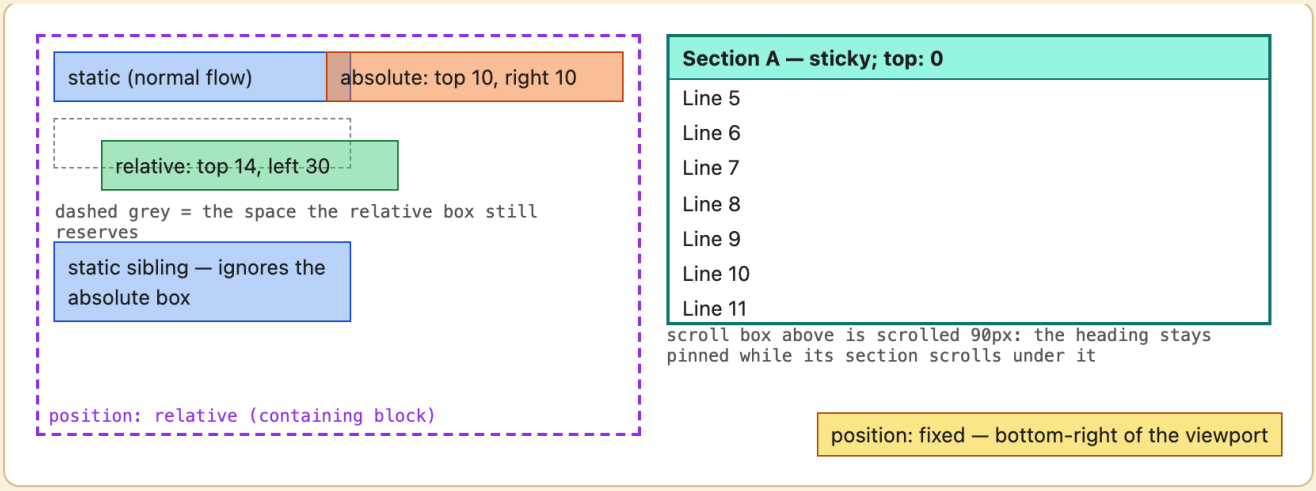


Figure 7.4 — `code/04_positioning.html`. The dashed grey rectangle is the space the relative box still reserves; the absolute box overlaps the static box because it no longer takes part in the flow.

The key CSS:

```
.container { position: relative; }                      /* becomes the containing block for .abs */
.rel  { position: relative; top: 14px; left: 30px; }      /* nudged; space kept */
.abs  { position: absolute; top: 10px; right: 10px; }     /* pinned to the container's corner */
.scroller { height: 180px; overflow: auto; }
.scroller h4 { position: sticky; top: 0; }                /* sticks to the scroll box's top */
.fixed { position: fixed; right: 12px; bottom: 12px; }    /* glued to the viewport */
```

Measured by `code/04_positioning.mjs` — including scrolling the page 500px to prove the fixed box does not move:

```
relative : moved 30px right, 14px down from its normal spot
absolute : 10px from container's right padding edge, 10px from its top
sticky   : heading top - scroller top = 0px after scrolling 90px (stays pinned)
fixed    : before scroll right/bottom gap = 12/12px, after scrolling 500px = 12/12px (glued to the viewport)
```

How each one works

`relative`

1. The box is laid out in normal flow.
2. It is then shifted visually by the insets. Neighbours do not move — they still see the original position.
3. Its most important job today is making itself the **containing block** for absolute children, usually with no offsets at all.

`absolute`

1. The box is removed from flow — siblings close up as if it weren't there.
2. The browser walks up the ancestors to the first one with `position` other than `static` (or with a `transform`, `filter`, `perspective`, `contain: layout/paint`, or `will-change` of those). Its **padding box** is the containing block. If there is none, the initial containing block (the size of the viewport, anchored at the top of the document) is used.
3. Insets position the box's margin edges relative to that rectangle. With `width: auto` the box **shrinks to fit** its content.
4. Setting both `left` and `right` (or `inset: 0`) stretches it; `inset: 0; margin: auto` plus a width and height centres it.

`fixed` Like absolute, but the containing block is the **viewport**, so it stays in place while the page scrolls. The trap: if any ancestor has a `transform`, `filter`, `perspective`, `backdrop-filter`, `contain: paint` or `will-change: transform`, that ancestor becomes the containing block instead, and your "fixed" modal scrolls away with it. That is one reason modals (dialogs that block the page behind them) are often rendered at the end of `<body>` (React "portals", which render a component somewhere else in the

DOM) — or, better, with the native `<dialog>` element, which uses the **top layer** (a special layer the browser paints above the whole page) and escapes all of this.

`sticky`

1. The box behaves as `relative` (in flow) until its scroll container scrolls it to the threshold you gave, e.g. `top: 0`.
2. Then it sticks at that offset — but **never leaves its parent**: when the parent's bottom edge scrolls up, the sticky box is pushed away with it. That is how "each section heading sticks until the next section" works.
3. It needs at least one inset (`top`, `bottom`…). Without one it never sticks.
4. It sticks to the nearest ancestor that is a **scroll container** (a box that can scroll its content: `overflow: auto/scroll/hidden`). If some wrapper has `overflow: hidden` without being the thing that actually scrolls, the sticky element sticks to that wrapper and appears not to work. (`overflow: clip` does not create a scroll container, so it is safe.)

Trade-offs

Approach	Good for	Beware
Absolute positioning for layout	Overlays, badges, decorative bits	Content can overlap and text can be cut off when it grows or when users zoom; not for main page layout
Fixed headers/footers	Always-available controls	Covers content (use `scroll-padding-top` so anchor links — links to an `#id` on the same page — don't hide under a fixed header); eats space on small screens
Sticky	Headers that should stay visible while scrolling	Silent failures from ancestor `overflow` or a parent that is exactly as tall as the sticky element

Common mistakes

- `position: absolute` with no positioned ancestor — the box jumps to the page's corner. Add `position: relative` to the intended parent.
- Using `top: 50%; left: 50%` to centre and forgetting `translate: -50% -50%` (the offsets move the *top-left corner* to the centre). Today prefer `inset: 0; margin: auto` or grid.
- Sticky inside a parent that is only as tall as the sticky element → there is no room to stick.
- Expecting `z-index` to work on a static element (7.8).

In the interview

30-second answer: "`static` is normal flow. `relative` keeps the element's space and shifts it visually; mostly it's used as the reference for absolute children. `absolute` removes it from flow and positions it against the nearest positioned ancestor's padding box. `fixed` positions against the viewport so it doesn't scroll — unless an ancestor has a transform or filter, which becomes the containing block. `sticky` is in flow until it hits a threshold in its scroll container, then it sticks, but only within its parent."

- *Why does my sticky header not stick?* → No `top`, an ancestor with `overflow` that isn't the scroller, or the parent is too short.
- *Why is my `position: fixed` modal scrolling with the page?* → An ancestor has `transform`/`filter`/`will-change`; move it out, or use `<dialog>`.
- *What's the containing block of an absolute element?* → The padding box of the nearest ancestor with non-static position (or a transform etc.); otherwise the initial containing block.

Self-check

1. Does a `position: relative; top: 20px` element move its next sibling down? → No; the sibling sees the original position.

2. An absolute child has `right: 0` in a `position: relative` parent with `padding: 20px; border: 5px`. Where is it? → Flush with the inner edge of the border (the padding box's right edge), not 20px in.
3. Which `overflow` value can you set on an ancestor without breaking `position: sticky`? → `visible` or `clip`.

7.7 Overflow

What it is

Overflow is content that does not fit its box — typically because you fixed a `height`, or a long word or wide image exceeds a `width`. The `overflow` property (or `overflow-x` / `overflow-y`) decides what happens:

Value	Overflowing content	Scrollbars	Scrollable by script?	Creates a BFC / scroll container?
`visible` (default)	Spills out and paints over what follows	No	No	No
`hidden`	Clipped (cut off, not drawn) at the padding box	No	**Yes** (`el.scrollTop = …`, focus, anchor links)	Yes / yes
`clip`	Clipped	No	**No**	No / no
`scroll`	Clipped	**Always**, even if not needed	Yes	Yes / yes
`auto`	Clipped	Only when needed	Yes	Yes / yes

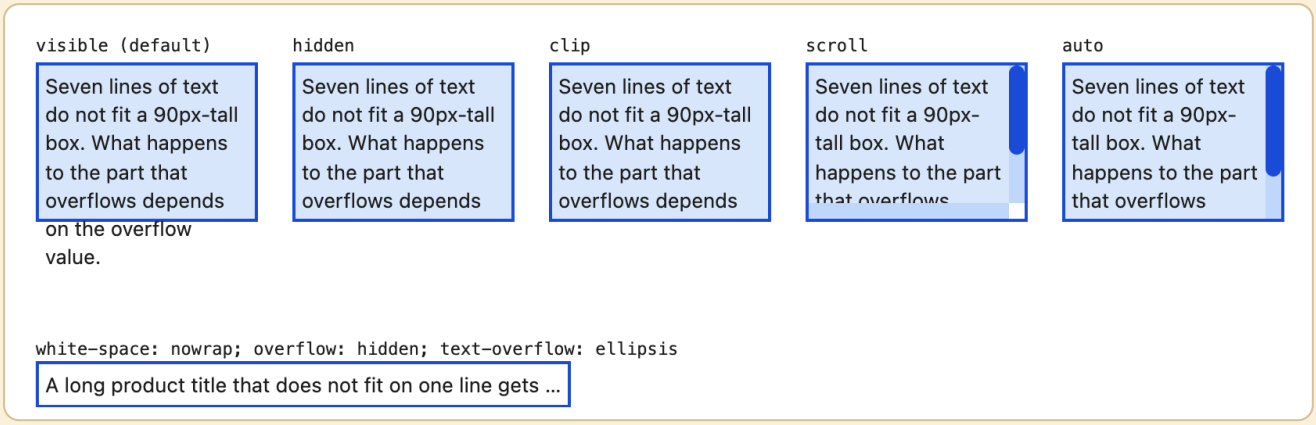


Figure 7.5 — `code/05_overflow.html`. The scrollbars are coloured with `::-webkit-scrollbar` so they show up in the screenshot; macOS normally uses overlay scrollbars (thin bars drawn over the content) that only appear while scrolling.

`code/05_overflow.mjs` sets `scrollTop = 20` on each box and reads it back:

```
visible  clientHeight 98  scrollHeight 131  scrollTop after setting 20 -> 0
hidden   clientHeight 98  scrollHeight 135  scrollTop after setting 20 -> 20
clip     clientHeight 98  scrollHeight 131  scrollTop after setting 20 -> 0
scroll   clientHeight 88  scrollHeight 135  scrollTop after setting 20 -> 20
auto     clientHeight 98  scrollHeight 135  scrollTop after setting 20 -> 20
```

Three things to read from this: `hidden` boxes *can* still be scrolled programmatically (by code) (a keyboard user tabbing to a link inside will scroll it too), `clip` cannot; `overflow: scroll` spent 10px of height on a horizontal scrollbar it didn't need (clientHeight 88); and the scroll containers count their bottom padding in the scrollable area (135 vs 131). `overflow: clip` reached Baseline in 2022.

Useful companions

- **Single-line truncation** — all three are required:

```
.title { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
```

- **Multi-line truncation**: `display: -webkit-box; -webkit-box-orient: vertical; -webkit-line-clamp: 3; overflow: hidden;` — prefixed (the `-webkit-` vendor prefix marks a property that started as one browser's own extension), but works in all current browsers.
- **Long words and URLs** breaking out of narrow columns: `overflow-wrap: anywhere` (or `break-word`).
- **Mixing axes**: if one axis is `visible` and the other isn't, `visible` computes to `auto` — you cannot clip only vertically while letting content spill horizontally (use `clip` for that).
- `scrollbar-gutter: stable` reserves space for a scrollbar so content doesn't jump when one appears.
- `overscroll-behavior: contain` stops scrolling inside a modal or chat panel from "chaining" to the page behind it (carrying on and scrolling the page once the panel reaches its end).

Common mistakes

- Using `overflow: hidden` to fix a float or margin problem and later finding focus outlines and box-shadows clipped.
- Putting `overflow: hidden` on `body`/`html` to lock scrolling and breaking `position: sticky` below it.
- Truncating text that the user needs to read with no way to see the rest (add a tooltip — a label that pops up on hover — or an expand control, and keep the full text in the DOM for screen readers).

Self-check

1. Which value clips without making the element scrollable? → `clip`.
2. You need an ellipsis on a single-line product title. Which three declarations? → `white-space: nowrap; overflow: hidden; text-overflow: ellipsis`.
3. Why can `overflow: scroll` look worse than `auto` on Windows? → It always shows scrollbars, even when nothing overflows.

7.8 `z-index` and stacking contexts

What it is

Boxes can overlap (negative margins, positioning, transforms, grid items in the same cell). The **stacking order** decides which one is drawn in front — the imaginary **z-axis** points out of the screen towards you. `z-index` sets an element's position on that axis, but only **within its stacking context**.

A **stacking context** is a group of elements that is painted as one unit. Analogy: each stacking context is a sheet of paper with drawings on it. You can reorder drawings *on* a sheet with `z-index`, and you can reorder whole sheets, but a drawing can never slip between two other sheets — no matter how big its number is.

How it works

1. `z-index` applies to **positioned** elements (`position` not `static`) and to **flex and grid items**. On a static block it does nothing.
2. Elements with a higher `z-index` in the **same** stacking context paint in front; equal values paint in DOM order (later in front).
3. An element with `z-index` set to a number (even `0`) on a positioned or flex/grid item **creates a new stacking context** for its descendants.
4. Its descendants' z-indexes are only compared with each other; to the outside world the whole group has its parent's z-index.

Inside one stacking context, the painting order is (back to front):

1. The context root's background and borders
2. Descendants with **negative** `z-index`
3. In-flow, non-positioned block boxes

4. Floats
5. Inline content (text, inline boxes)
6. Positioned descendants with `z-index: auto` or `0`, in DOM order
7. Positioned descendants with **positive** `z-index`, lowest first

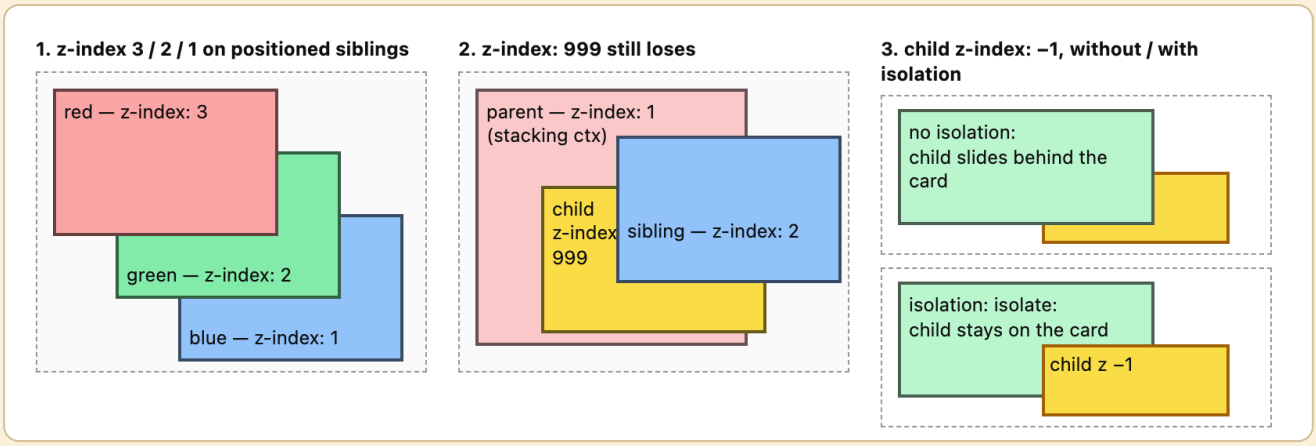


Figure 7.6 — `code/06_stacking.html`.

```
/* demo 2 — the classic trap */
.parent  { position: absolute; z-index: 1; }     /* creates a stacking context */
.child   { position: absolute; z-index: 999; }   /* 999 only counts INSIDE .parent */
.sibling { position: absolute; z-index: 2; }     /* 2 > 1, so it covers .parent and everything in it */

/* demo 3 — isolation: isolate makes a stacking context without z-index */
.card.iso { isolation: isolate; }                /* the child's z-index: -1 now stays above the card's background
*/
```

`code/06_stacking.mjs` asks the browser `document.elementFromPoint(x, y)` — "which element would receive a click here?" — at points where the boxes overlap:

```
demo 1  where red, green and blue overlap -> r1
demo 2  where child (999) and sibling (2) overlap -> b2
demo 3  inside the yellow child, no isolation -> card
demo 3  inside the yellow child, isolation: isolate -> u2
```

What creates a stacking context

- The root element (`<html>`)
- `position: relative`/`absolute` **with** a `z-index` other than `auto`
- `position: fixed` or `sticky` (always)
- A flex or grid item with `z-index` other than `auto`
- `opacity` less than 1
- `transform`, `scale`, `rotate`, `translate`, `filter`, `backdrop-filter`, `perspective`, `clip-path`, `mask` (any value other than `none`)
- `mix-blend-mode` other than `normal`
- `isolation: isolate`
- `will-change` naming any property above
- `contain: layout`, `paint`, `strict` or `content`; `container-type: size` or `inline-size`
- Elements in the **top layer** — an open modal `<dialog>`, a shown popover (a small floating panel created with the `popover` attribute), a fullscreen element — are painted above *everything*, whatever the z-indexes

Why it matters

"I set `z-index: 99999` and the dropdown is still behind the header" is almost always a stacking-context problem: an ancestor of the dropdown (perhaps with `transform` for an animation, or `opacity: .99`) created a context whose own level is below the header's.

How to debug and fix it

1. Walk up from the element that is "behind" and look for any ancestor in the list above (DevTools → Layers panel in Chrome/Edge, or the 3D view in Edge, visualises contexts).
2. Compare the z-indexes of the **two ancestors that are siblings in the same context**, not the elements themselves.
3. Fix by raising the ancestor's z-index, removing the property that created the unwanted context, or moving the overlay elsewhere in the DOM (or into the top layer with `<dialog>` / the `popover` attribute).
4. Prevent it: keep a small **z-index scale** as custom properties (CSS variables, Chapter 6) (`--z-dropdown: 100; --z-sticky: 200; --z-modal: 300;`) and use `isolation: isolate` on components so their internal z-indexes cannot leak out.

In the interview

30-second answer: "`z-index` only orders elements within the same stacking context, and only works on positioned or flex/grid items. Things like a z-index on a positioned element, opacity below 1, transforms, filters and `isolation: isolate` create a new context, and everything inside is flattened into it — so a child with `z-index: 999` can't escape a parent with `z-index: 1` that sits below a sibling with `z-index: 2`. To debug I find the ancestors that are siblings in a common context and compare those."

- *Does `z-index` work on a static element?* → No (except flex and grid items).
- *What's the difference between `z-index: auto` and `0`?* → Both paint at the same level, but `0` creates a stacking context and `auto` doesn't.

Self-check

1. Name three properties besides `z-index` that create a stacking context. → `opacity < 1`, `transform`, `filter` (also `isolation`, `will-change`, `mix-blend-mode`…).
2. Parent A `z-index: 5` contains child `z-index: -1`. Can the child appear behind A's background? → No; A is a stacking context, so its own background is always at the very back of that context.
3. Why does an open `<dialog>` appear above a `z-index: 10000` header? → Modal dialogs are in the top layer, above all stacking contexts.

7.9 Floats (legacy)

What it is

`float: left` or `float: right` takes a box out of normal flow, pushes it to that side of its container, and lets **inline content (text) wrap around it**. It was designed for magazine-style images inside articles. From roughly 2005 to 2015 floats were also abused to build whole page layouts (columns, grids), with a pile of hacks to go with it. Flexbox and grid replaced that; you still need floats to wrap text around a shape, and to read older code.

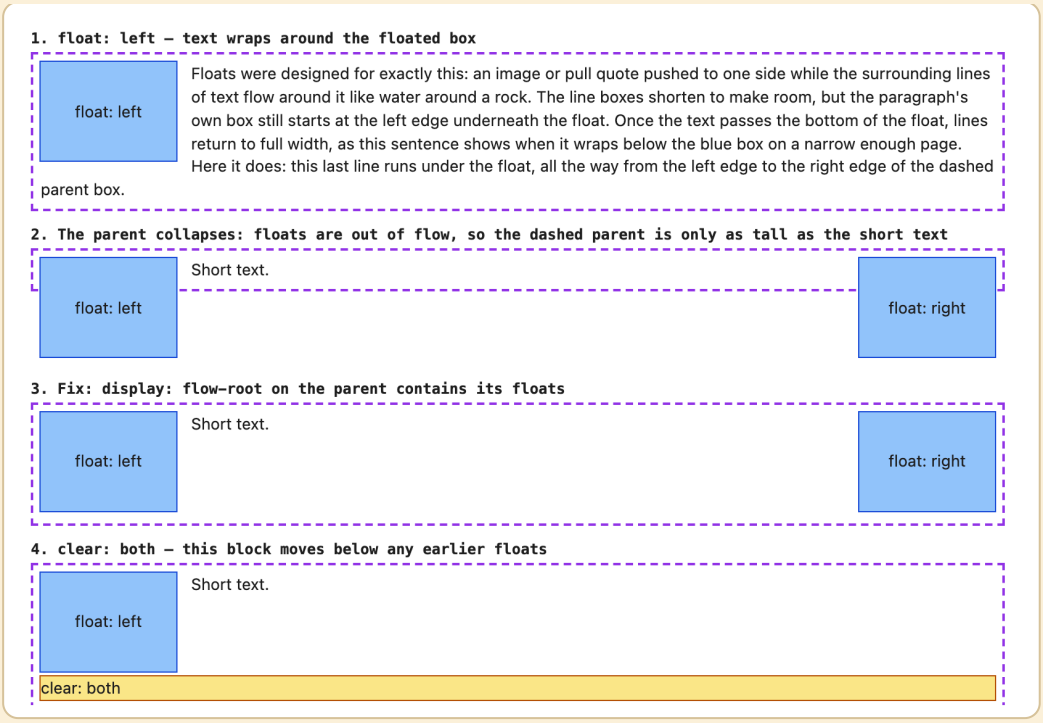


Figure 7.7 — `code/07_floats.html`.

How it works

1. The float is moved as far left (or right) as it can go within its containing block, at the vertical position where it would have been.
2. Following block boxes act as if the float isn't there — but their **line boxes shorten** to avoid it, so text flows around it.
3. The parent's height ignores floats — it **collapses** if everything inside is floated (demo 2).
4. `clear: left | right | both` on a later block forces it to start below the earlier floats (demo 4).
5. A parent that establishes a **BFC** — best done with `display: flow-root` — grows to contain its floats (demo 3).

Measured parent heights (each dashed parent has `padding: 6px` and a 2px border; the floats are 80px tall with a 4px bottom margin):

```
w1  display: block      parent height 129.0625px
w2  display: block      parent height 34.84375px
w3  display: flow-root  parent height 100px
w4  display: flow-root  parent height 118.84375px
```

Output of `code/07_floats.mjs`. `w2` is only as tall as one line of text plus its padding and border; `w3`, with identical content, contains the floats: 80 + 4 (float margin) + 2×6 (padding) + 2×2 (border) = **100px**.

The clearfix (read it, don't write it)

Before `flow-root`, the standard way to contain floats was this pseudo-element hack; you will still see it in legacy code and Bootstrap 3–era CSS (Bootstrap is a popular CSS framework; version 3 dates from 2013):

```
.clearfix::after { content: ""; display: table; clear: both; }
```

When to use floats today

Need	Use
Wrap paragraph text around an image or pull quote (a quotation repeated in large type)	`float` (optionally `shape-outside: circle(50%)` to wrap around a shape)
Columns, rows of cards, nav bars, sidebars	Flexbox or grid
Push one item to the right in a row	Flexbox `margin-left: auto` (Chapter 8)

Note that `float` is ignored on flex and grid items — another reason floats and modern layout don't mix.

Self-check

1. Why does a parent with only floated children have zero height? → Floats are out of flow, so they don't contribute to the parent's auto height.
2. Modern one-liner to contain floats? → `display: flow-root` on the parent.
3. What does `clear: both` do? → Moves the element below any preceding left or right floats.

Chapter 7 TL;DR

- **Box model**: content → padding → border → margin. Margin is transparent and outside; background covers content + padding (and under the border).
- `box-sizing: border-box` makes `width` include padding and border; put it on `*, *::before, *::after` in every project. Content-box: 200 + 2×20 + 2×6 = 252.
- **Percent padding/margin** (even vertical) use the containing block's **width**. `width: auto` ≠ `100%`.
- **Margin collapsing**: vertical margins in block flow merge to the largest (positive + most negative if mixed); happens between siblings, parent/first-last child, and empty blocks; never in flex/grid, for floats/absolute, or across padding/border/new BFC.
- `display`: outer (block/inline) + inner (flow/flow-root/flex/grid). Inline ignores width/height and vertical margins; inline-block keeps them. `none` removes the box; `contents` removes only the element's own box (check accessibility).
- **Normal flow**: blocks stack in the block direction, inlines wrap into line boxes. A **BFC** (`display: flow-root`) contains floats and stops margin collapse through its edges.
- **Position**: `relative` = shifted, space kept; `absolute` = out of flow, against nearest positioned ancestor's padding box; `fixed` = viewport (unless an ancestor has transform/filter); `sticky` = in flow until its threshold in the nearest scroll container, never leaves its parent.
- **Overflow**: visible / hidden (script-scrollable) / clip (not scrollable, no BFC) / scroll (always bars) / auto (bars when needed). Ellipsis needs `nowrap + hidden + text-overflow`.
- **Stacking**: `z-index` works on positioned and flex/grid items, only within its **stacking context**; contexts come from z-index on positioned elements, opacity < 1, transforms, filters, `isolation: isolate`, fixed/sticky and more. The top layer beats everything.
- **Floats**: for wrapping text around images; contain with `flow-root`, escape with `clear`; don't build layouts with them.

Next: Chapter 8 — Flexbox.